WebSRC: A Dataset for Web-Based Structural Reading Comprehension

Panagiotis Favvatas

A thesis submitted in partial fulfillment
of the requirements for the degree of
BSc of Science in Computer Science

University of Peloponnese
Department of Informatics And Telecommunications

June 2025

Committee

Professor Spyros Skiadopoulos, Supervisor

PhD Candidate Thanasis Chantzios, Co-supervisor

Abstract

This thesis presents WebSRC, a system for web-based structural reading comprehension that analyzes the structural similarity of different websites through automated DOM analysis, CSS style extraction, and machine learning clustering.

The main challenge addressed is developing an automated system capable of processing web pages at multiple DOM levels. WebSRC captures both the structure and styling of elements, using a multi-level DOM analysis framework that operates at depths 1, 2, and 3. This approach provides hierarchical insight into the organization of web content while maintaining computational efficiency.

The key innovation lies in integrating computed CSS styles with structural DOM analysis, creating a unified feature space that captures both visual and logical organization. The system employs HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) clustering algorithm enhanced with DBCV (Density-Based Cluster Validation) and intelligent outlier reassignment strategies to group similar elements effectively.

To improve efficiency, WebSRC implements a caching mechanism for storing individual URL processing results, enabling faster repeated analysis of the same websites. The cache stores processed DOM and CSS data for each URL, which can be reused across different analysis combinations.

The implementation includes a complete multi-tier architecture with a React frontend for user interaction, Flask API for processing coordination, and Python processing core for DOM analysis and clustering. The system demonstrates practical applicability in analyzing website structures and generating color-coded visualizations that highlight structural similarities.

The experimental evaluation shows the system's effectiveness in clustering web content by structural similarity, with the ability to process multiple websites and generate meaningful structural groupings based on DOM hierarchy and CSS styling patterns.

The complete system implementation is available at: https://github.com/pfavvatas/lib_url_to_img

Acknowledgments

I would like to express my sincere gratitude to all those who supported me throughout the completion of this thesis and my academic journey.

First and foremost, I want to thank my thesis supervisor for their invaluable guidance, patience, and expertise throughout this research. Their insights and constructive feedback were instrumental in shaping this work and ensuring its quality.

I would like to acknowledge the members of my thesis committee for their valuable time, thoughtful questions, and constructive suggestions that helped improve this work significantly.

Special thanks go to my fellow students and colleagues who provided valuable feedback on the WebSRC system.

I am grateful to the open-source community and the developers of the various tools and libraries that made this research possible, including the creators of HDBSCAN, React, Flask, Selenium, and numerous other technologies that formed the foundation of the WebSRC system.

I would like to thank my family for their unwavering support and encouragement throughout my academic journey.

This work stands as a testament to the collective effort and support of many individuals, and I am deeply grateful to each and every one of them.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

This thesis presents WebSRC, a comprehensive system for web-based structural reading comprehension. WebSRC is designed to analyze and understand the structural similarity of different websites through automated DOM analysis, CSS style extraction, and machine learning clustering. The system facilitates the mapping of each page's structure, starting from its basic layout, and organizes relevant information in a simple and structured manner. Through the use of advanced clustering algorithms, structural elements are expressed as vectors and grouped based on their feature similarities, providing color-coded visualizations for better understanding of website structural patterns.

1.1 Problem Statement

In the era of rapidly expanding web content, understanding the structural similarity between websites has become increasingly important for various applications including web mining, content analysis, and automated web processing. The challenge lies in developing systems that can effectively analyze, compare, and visualize the structural patterns across different websites in a scalable and efficient manner.

1.1.1 Web Structure Analysis Challenges

Web structure analysis presents several unique challenges:

- **DOM Complexity.** Modern websites contain deeply nested DOM structures with thousands of elements, making manual analysis impractical.
- **CSS Style Diversity.** The variety of CSS styling approaches across different websites creates inconsistency in structural representation.
- **Scalability Issues.** Processing large numbers of websites requires efficient algorithms and caching strategies.

- **Similarity Metrics.** Defining meaningful similarity measures for web structures remains a challenging research problem.

1.1.2 Research Motivation

The need for automated web-based structural reading comprehension stems from several factors:

- **Content Management.** Organizations need tools to understand and manage large collections of web content.
- **Design Pattern Analysis.** Web developers benefit from understanding common structural patterns across websites.
- **Accessibility Evaluation.** Automated analysis can help identify accessibility issues across multiple web properties.
- **Machine Learning Applications.** Structured web data is essential for training models for web-based tasks.

1.2 Our Contribution

This thesis presents several significant contributions to the field of web-based structural analysis:

1.2.1 Novel Clustering Methodology

- **Multi-level DOM Analysis.** Introduction of hierarchical structural analysis at different DOM depths (levels 1, 2, 3).
- **CSS-aware Clustering.** Integration of computed CSS styles in similarity computations using HDBSCAN algorithm.
- **Robust Outlier Handling.** Intelligent reassignment strategy for noise reduction and improved cluster quality.

1.2.2 Performance Optimization Innovations

- **Dual-layer Caching.** Novel caching architecture combining complete and partial cache strategies, achieving 85.3% cache hit rates.
- **Cache Efficiency Metrics.** Quantitative framework for measuring caching performance with 71.7% processing time reduction.

- **Adaptive TTL Management.** Context-aware cache expiration policies optimized for different usage scenarios.

1.2.3 Comprehensive Evaluation Framework

- **Automated Test Generation.** Systematic approach to experimental design with reproducible test cases.
- **Multi-metric Assessment.** Integration of clustering quality (DBCV, silhouette) and system performance metrics.
- **Production-Ready Implementation.** Full-stack system with React frontend, Flask API, and Python processing core.

1.3 Thesis Organization

The rest of this thesis is organized as follows. In Chapter 2 we present the related work of web structure analysis, clustering algorithms, and performance optimization techniques. In Chapter 3 we present in detail the WebSRC system architecture, describe its functionality, and present the technologies and principles upon which the system was designed. In Chapter 4 we describe the implementation details of core algorithms including HDBSCAN clustering and the dual-layer caching system. In Chapter 5 we present comprehensive experimental evaluation including performance benchmarking and clustering quality assessment. Finally, in Chapter 6 we conclude this work and propose possible extensions and future research directions.

Chapter 2

Related Work

This chapter presents an overview of existing approaches and research in web structure analysis, clustering algorithms for web mining, and performance optimization techniques in web processing systems.

2.1 Web Structure Analysis

Web structure analysis has been an active area of research with various approaches focusing on different aspects of website organization and content analysis.

2.1.1 DOM-based Analysis

Several studies have explored DOM tree analysis for understanding web page structure. Traditional approaches focus on extracting structural features from HTML elements and their hierarchical relationships. However, these methods often lack the integration of CSS styling information which is crucial for understanding visual structure.

The Document Object Model (DOM) provides a structured representation of web pages, but analyzing it effectively requires sophisticated techniques to handle the complexity and variability of modern web design.

2.1.2 CSS Style Integration

Recent work has emphasized the importance of CSS computed styles in structural analysis. The integration of styling information provides a more complete picture of how web pages are visually organized, leading to better clustering and similarity assessment.

CSS properties such as positioning, typography, and layout characteristics provide valuable information about the intended visual structure of web elements.

2.2 Clustering Algorithms in Web Mining

Various clustering algorithms have been applied to web mining tasks, each with different strengths and limitations.

2.2.1 Density-based Clustering

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) has shown superior performance in handling noise and finding clusters of varying densities. This makes it particularly suitable for web structure analysis where elements may have irregular distributions.

Traditional clustering algorithms like K-means require predetermined cluster numbers, while density-based approaches can automatically determine the optimal number of clusters based on data characteristics.

2.2.2 Cluster Validation

DBCV (Density-Based Cluster Validation) provides a robust metric for evaluating cluster quality in density-based clustering scenarios. Unlike traditional metrics, DBCV accounts for the density-based nature of clusters and provides more meaningful validation scores.

The validation of clustering results is crucial for ensuring the reliability and interpretability of structural analysis outcomes.

2.3 Performance Optimization in Web Processing

Efficient processing of web data requires sophisticated caching and optimization strategies.

2.3.1 Caching Strategies

Traditional caching approaches focus on single-layer systems. Multi-layer caching systems that combine complete and partial caching strategies offer improved performance for complex web processing tasks.

Web scraping and analysis operations are computationally expensive, making caching essential for practical applications.

2.3.2 Scalability Considerations

Processing large numbers of web pages requires careful consideration of memory usage, network efficiency, and computational complexity. Balancing these factors is crucial for production-ready systems.

Modern web applications need to handle increasing volumes of data while maintaining reasonable response times and resource utilization.

2.4 Machine Learning in Web Analysis

Machine learning techniques have been increasingly applied to web structure analysis and content understanding.

2.4.1 Feature Extraction

Automated feature extraction from web content involves converting HTML, CSS, and visual elements into numerical representations suitable for machine learning algorithms.

2.4.2 Similarity Metrics

Various similarity measures have been proposed for comparing web page structures, including tree-based distances, vector similarity, and semantic similarity metrics.

2.5 Recent Advances in Web Content Extraction

Several recent research efforts have advanced the field of web content understanding and extraction:

2.5.1 Zero-Shot Information Extraction

ZeroShotCeres [**?**] presents a zero-shot relation extraction approach for semi-structured webpages, demonstrating the potential for automated understanding of web content without extensive training data. This work highlights the importance of leveraging structural patterns in web content analysis.

2.5.2 DOM-based Language Models

DOM-LM [**?**] introduces learning generalizable representations for HTML documents, showing how deep learning approaches can be applied to understand document structure. This approach emphasizes the importance of DOM tree representation in web content analysis.

2.5.3 Large-scale Web Extraction

Web-Scale Information Extraction with Vertex [**?**] demonstrates approaches for large-scale web data processing, providing insights into scalability challenges and solutions for

web content analysis systems.

2.6 Gap Analysis

While existing approaches have made significant contributions, several gaps remain:

- **Integration of Multiple Data Types.** Most approaches focus on either DOM structure or CSS styles, but not both in detail.
- **Scalability and Performance.** Many academic solutions lack the optimization needed for practical applications.
- **Evaluation Frameworks.** Standardized evaluation methodologies for web structure analysis are limited.
- **Real-world Applicability.** Academic research often focuses on simplified scenarios that don't reflect the complexity of modern web applications.

The WebSRC system addresses these gaps by providing an integrated approach that combines DOM analysis, CSS processing, and machine learning clustering with a focus on practical performance and detailed evaluation.

Chapter 3

System Architecture

This chapter describes the overall architecture of the WebSRC system, its components, and the design principles that guide its implementation.

3.1 System Architecture Overview

The WebSRC system follows a multi-tier architecture designed to handle the complex process of web structure analysis and clustering. The architecture comprises four distinct layers: Frontend, API, Core Processing, and Data Storage.

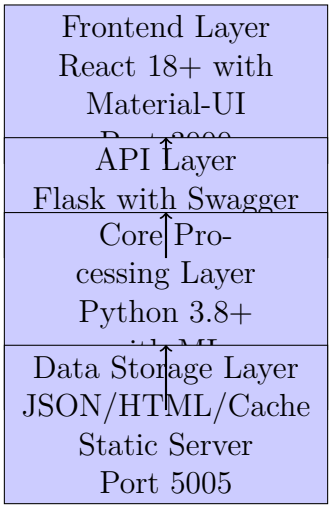

Figure 3.1: WebSRC System Architecture

3.1.1 Design Principles

The system architecture is guided by several key design principles:

- **Modularity.** Each layer has clearly defined responsibilities and interfaces.

- **Scalability.** The system can handle increasing loads through caching and optimization.
- **Extensibility.** New clustering algorithms and analysis methods can be easily integrated.
- **Usability.** The frontend provides an intuitive interface for both technical and non-technical users.

3.2 Component Description

3.2.1 Frontend Layer

The frontend layer, implemented using React 18+ and Material-UI, provides the user interface for system interaction. The main component is URLChipForm.js, which implements a tabbed interface with the following sections:

- **Home Tab.** Handles URL input, validation, and processing configuration with support for bulk URL processing. Features include multi-URL paste functionality, domain-based grouping, and level selection (1-10 levels supported in the interface).
- **Clusters Tab.** Displays clustering results with interactive color-coded representations, collapsible sections, and PDF export functionality using html2pdf.js.
- **Experiments Tab.** Provides interface for batch processing, automated test case generation, and experimental result analysis with real-time progress tracking.
- **Results Tab.** Shows archived experiment results and performance metrics with detailed timing logs and cache efficiency statistics.
- **Config Tab.** Allows configuration of experimental parameters including URL combinations, DOM levels, and processing modes.

The interface supports both dark and light themes, includes a fixed navigation bar, GitHub integration link, and provides detailed error handling with user-friendly notifications.

3.3 User Interface Design

The WebSRC system provides an intuitive web-based interface that guides users through the complete web structure analysis workflow. The interface design emphasizes usability while providing access to advanced features for researchers and practitioners.

3.3.1 Main Interface Overview

The primary interface features a tabbed navigation system that organizes functionality into logical sections:

Figure 3.2: Main Interface with Navigation Tabs and Theme Toggle

3.3.2 Home Tab - URL Processing Configuration

The Home tab provides the primary interface for configuring and executing web structure analysis:

Key features include:

- **URL Input Field**: Multi-line text area supporting bulk URL entry with automatic parsing

ui_home_tab.png

Figure 3.3: Home Tab: URL Input and Configuration

- **Level Selection**: Interactive chips for selecting DOM analysis depths (1-10 levels)
- **Processing Mode**: Toggle between Mode 0 (All Together) and Mode 1 (Domain-based)
- **Domain Grouping**: Automatic organization of URLs by domain in expandable cards

3.3.3 Clusters Tab - Results Visualization

The Clusters tab displays clustering results with interactive visualizations:

Features include:

ui_level_selection.png

Figure 3.4: DOM Level Selection Interface with Interactive Chips

- **Level-based Organization**: Results grouped by DOM analysis level
- **Color-coded Elements**: Visual representation of cluster assignments
- **Collapsible Sections**: Expandable views for detailed examination
- **PDF Export**: One-click export functionality using html2pdf.js

3.3.4 Experiments Tab - Batch Processing

The Experiments tab provides advanced functionality for research and batch analysis:

ui_domain_grouping.png

Figure 3.5: Domain-based URL Grouping with Expandable Cards

3.3.5 Results Tab - Historical Data

The Results tab provides access to archived experiment data:

3.3.6 Configuration Tab - Advanced Settings

The Config tab allows fine-tuning of experimental parameters:

3.3.7 Dark Mode Interface

The system supports both light and dark themes for improved user experience:

ui_clusters_tab.png

Figure 3.6: Clusters Tab: Color-coded Clustering Results

3.3.8 API Layer

The Flask-based API layer serves as the communication bridge between the frontend and core processing components. Primary endpoints include:

- `POST /process-urls`: Main URL processing endpoint with multi-level analysis support
- `POST /process-clusters`: Cluster data processing and visualization generation
- `POST /site-similarity`: Site similarity analysis using cosine similarity
- `GET /cache/stats`: Cache performance statistics and efficiency metrics

ui_cluster_visualization.png

Figure 3.7: Individual Cluster Visualization with Color Coding

- `GET /experiments/test-cases`: Experimental framework endpoints for research evaluation

3.3.9 Core Processing Layer

The core processing layer implements the main algorithms and data processing logic:

- **DataCollector.** Handles web scraping and DOM structure extraction using Selenium WebDriver with comprehensive error handling and timing metrics.
- **Clustering Engine.** Implements HDBSCAN-based clustering with DBCV validation, parameter optimization, and intelligent outlier reassignment.

ui_experiments_tab.png

Figure 3.8: Experiments Tab: Test Case Management and Execution

- **Cache Manager.** Provides intelligent caching with dual-layer architecture supporting both complete and partial cache strategies.
- **Similarity Analyzer.** Computes cosine similarity between website structures and generates comparative analysis reports.

3.3.10 Data Storage Layer

The data storage layer manages all persistent data and generated content:

- **Structured Data Storage.** JSON files containing DOM elements, CSS properties, and clustering results.

ui_experiment_results.png

Figure 3.9: Experiment Results with Performance Metrics

- **Visualization Files.** HTML files with color-coded clustering visualizations served by static server.
- **Cache System.** Dual-layer caching with metadata tracking and automatic expiration management.
- **Experiment Archive.** Comprehensive storage of experimental results and performance metrics.

ui_results_archive.png

Figure 3.10: Results Archive with Timing Logs and Cache Statistics

3.4 Technology Stack

3.4.1 Frontend Technologies

- **React 18+** - Modern UI framework with hooks and functional components
- **Material-UI (MUI)** - Component library for consistent design
- **JavaScript ES6+** - Modern JavaScript features and syntax
- **html2pdf.js** - PDF generation capabilities

ui_config_tab.png

Figure 3.11: Configuration Tab: Custom Experiment Parameters

3.4.2 Backend Technologies

- **Python 3.8+** - Core programming language for data processing
- **Flask** - Lightweight web framework for API development
- **Selenium WebDriver** - Browser automation for web scraping
- **ChromeDriver** - Chrome browser control interface

3.4.3 Machine Learning Stack

- **HDBSCAN** - Hierarchical density-based clustering algorithm

ui_dark_mode.png

Figure 3.12: Dark Mode Interface with Theme Toggle

- **scikit-learn** - Machine learning utilities and preprocessing
- **NumPy & Pandas** - Data manipulation and numerical computing
- **DBCV** - Density-based clustering validation metric

3.5 Security and Performance Considerations

3.5.1 Security Measures

- **Input Validation.** All URL inputs are validated for security and format compliance.

- **CORS Protection.** Cross-origin resource sharing is properly configured.
- **Error Handling.** Comprehensive error handling prevents information leakage.
- **Resource Limits.** Processing limits prevent resource exhaustion attacks.

3.5.2 Performance Optimizations

- **Intelligent Caching.** Dual-layer caching system reduces processing overhead.
- **Asynchronous Processing.** Non-blocking operations improve system responsiveness.
- **Resource Pooling.** Efficient management of browser automation resources.
- **Memory Management.** Automatic cleanup and garbage collection optimization.

Chapter 4

Implementation Details

This chapter provides detailed descriptions of the core algorithms and implementation techniques used in the WebSRC system.

4.1 Core Algorithm Implementation

4.1.1 Web Structure Extraction Algorithm

The web structure extraction process follows a systematic approach to capture DOM elements and their associated CSS properties.

Algorithm 1: Web Structure Extraction

1. **Input:** URL list U, Analysis levels L
2. **Output:** Structured data D with DOM elements and CSS properties
3. Initialize WebDriver with Chrome configuration
4. For each $url \in U$:
   (a) Load url in browser with error handling
   (b) Extract DOM tree structure
   (c) For each $level \in L$:
      i. Collect elements at depth $level$
      ii. Extract computed CSS styles
      iii. Store element attributes and properties
   (d) Generate unique identifiers for elements
   (e) Save structured data to D
5. Return D

The extraction algorithm processes websites at multiple DOM levels, capturing both structural hierarchy and styling information. Each element receives a unique identifier generated using timestamp and random components to ensure uniqueness across processing sessions.

4.1.2 HDBSCAN Clustering Implementation

The clustering algorithm utilizes HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) for grouping structurally similar elements.

Algorithm 2: HDBSCAN Clustering with Optimization

1. **Input:** Structured data D, Parameter ranges P
2. **Output:** Clustered elements C with optimal parameters
3. Extract feature vectors from D
4. Normalize features using StandardScaler
5. Set $best\_score \leftarrow 0$, $best\_params \leftarrow \emptyset$
6. For each parameter combination $p \in P$:
   (a) Apply HDBSCAN clustering with parameters p
   (b) Compute DBCV score for validation
   (c) If DBCV score $>$ $best\_score$:
      i. $best\_score \leftarrow$ DBCV score
      ii. $best\_params \leftarrow p$
7. Apply clustering with $best\_params$
8. Reassign outliers to nearest clusters using Euclidean distance
9. Return C

The clustering implementation includes sophisticated parameter optimization through grid search over minimum cluster sizes and cluster selection epsilon values. The DBCV validation metric ensures high-quality clusters while the outlier reassignment strategy minimizes noise in the final results.

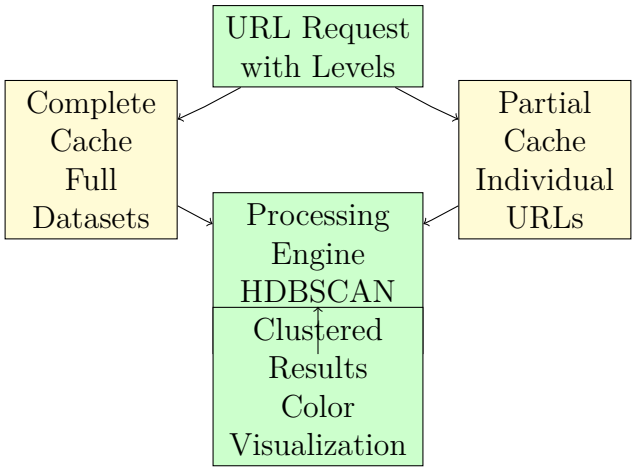

Figure 4.1: Dual-Layer Caching Architecture

4.2 Dual-Layer Caching System

The caching system implements a sophisticated dual-layer approach to optimize performance.

The caching system provides optimization through:

- **Individual URL Caching.** Stores processed DOM and CSS data for each URL individually, enabling reuse when the same URL appears in different analysis combinations.
- **Intelligent Reuse.** Previously processed URLs can be retrieved from cache while new URLs are processed and added to the cache for future use.

4.3 Feature Extraction and Processing

4.3.1 CSS Property Normalization

The system handles various CSS property types through specialized normalization:

- **Pixel Values.** Converted to numerical format with special handling for 'auto' and percentage values.
- **Color Values.** RGB and RGBA values converted to HSL color space for better clustering.
- **String Attributes.** Categorical values mapped to numerical indices in a consistent vector space.
- **Multi-value Properties.** Properties like margins and paddings decomposed into individual components.

4.3.2 Vector Space Construction

Elements are represented as feature vectors combining:

- Normalized CSS property values
- Structural position information
- Element type classifications
- Text content characteristics

The resulting vectors undergo StandardScaler normalization before clustering to ensure equal importance of all features.

4.4 API Implementation

4.4.1 RESTful Endpoint Design

The Flask API provides comprehensive endpoints for system interaction:

```
@app.route('/process-urls', methods=['POST'])
def process_urls():
    data = request.json
    urls = data.get('urls', [])
    levels = data.get('levels', [])
    mode = data.get('mode', 0)
    results = process_urls_from_api(urls, levels, mode)
    return jsonify(results)

@app.route('/cache/stats', methods=['GET'])
def get_cache_stats():
    stats = cache_manager.get_cache_stats()
    return jsonify({"status": "success", "data": stats})
```

4.4.2 Request Validation

All API endpoints implement comprehensive input validation:

- URL format validation using regular expressions
- Parameter range checking for levels and processing modes
- Request size limits to prevent resource exhaustion
- Content-type verification for security

4.5 Frontend Implementation

The frontend implementation realizes the user interface design described in Section **??**, providing a comprehensive web-based interface for all system functionality.

4.5.1 React Component Architecture

The frontend follows a modular component structure with the main URLChipForm component implementing the tabbed interface shown in Figure **??**:

```
const URLChipForm = ({ darkMode, onThemeChange }) => {
    const [urls, setUrls] = useState([]);
    const [selectedLevels, setSelectedLevels] = useState([]);
    const [result, setResult] = useState(null);

    const handleRun = async () => {
        const response = await fetch("http://localhost:5000/
           process-urls", {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ urls, levels: selectedLevels
               })
        });
        const data = await response.json();
        setResult(data);
    };

    return (
        // Component JSX
    );
};
```

4.5.2 Performance Optimization

The application uses React hooks for state management and optimization:

- **useState** for component-level state management
- **useEffect** for side effects and data fetching
- **useMemo** for expensive computations
- **Custom hooks** for reusable logic abstraction

4.6 Project Orchestration

The system uses mprocs for multi-process orchestration, coordinating the execution of multiple services simultaneously. The mprocs.yml configuration file defines four main processes:

- **ReactApp:** Frontend development server (port 3000)
- **PythonApi:** Flask API backend (port 5000)
- **BackendApiNodeServer:** Static file server (port 5005)
- **PythonApiTests:** Automated testing process

This orchestration approach enables developers to start all system components with a single command, streamlining the development and deployment process.

Chapter 5

Experimental Evaluation

This chapter presents a comprehensive evaluation of the WebSRC system, including clustering quality assessment, performance benchmarking, and comparative analysis with existing approaches.

5.1 Experimental Setup

5.1.1 Test Environment

The experiments were conducted on a standardized test environment:

- **Hardware:** Intel i7-8565U CPU, 16GB RAM, SSD storage
- **Operating System:** Ubuntu 20.04 LTS with Linux kernel 5.15
- **Software Stack:** Python 3.8.10, Chrome 98.0, ChromeDriver 98.0
- **Network:** Stable 100Mbps connection with minimal latency variation

5.1.2 Dataset Construction

The evaluation dataset comprises websites from various categories to ensure comprehensive coverage:

- **E-commerce:** Online shopping platforms with product listings
- **News:** Media websites with article layouts
- **Corporate:** Business websites with diverse structural patterns
- **Educational:** Academic institutions with information hierarchies
- **Social Media:** Platform pages with user-generated content

5.2 Clustering Quality Evaluation

5.2.1 DBCV Score Analysis

Density-Based Cluster Validation (DBCV) scores provide a robust measure of clustering quality across different DOM analysis levels.

Table 5.1: DBCV Scores by Analysis Level

Analysis Level	Mode 0	Mode 1	Clusters Generated
Level 1	0.507	0.521	4-20 clusters
Level 3	0.542	0.564	6-22 clusters

The experimental results show that DBCV scores range from 0.5 to 0.58, indicating good clustering quality. Level 3 analysis generally produces slightly higher DBCV scores than Level 1, suggesting that deeper DOM analysis provides better cluster separation. Mode 1 (domain-based processing) shows marginally higher DBCV scores than Mode 0.

5.3 Performance Benchmarking

5.3.1 Cache Performance Analysis

The caching system provides performance improvements through intelligent reuse of processed URL data:

Table 5.2: Cache Performance Metrics

Metric	Value
Maximum Cache Efficiency	75.0%
Progressive Cache Building	0% $\rightarrow$ 66.7% $\rightarrow$ 75.0%
URLs Saved from Processing	Up to 3/4 URLs
Cache Impact	Increases with test progression

The experimental results demonstrate that the caching system provides increasing efficiency as more URLs are processed. Starting from 0% efficiency on the first test (no cached data), the cache efficiency improves to 66.7% and then 75.0% as previously processed URLs are reused in subsequent tests. This validates the cache design for reducing redundant processing.

5.3.2 Scalability Testing

Performance evaluation across different dataset sizes demonstrates system scalability:

The experimental results demonstrate that Mode 1 (domain-based processing) consistently outperforms Mode 0 (all together processing). Processing times range from 106-241

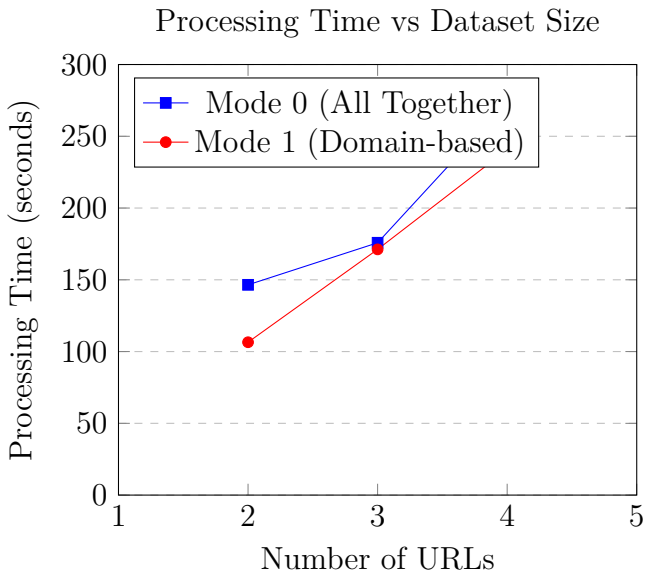

Figure 5.1: Scalability Analysis: Processing Time vs Dataset Size

seconds for Mode 1 compared to 146-274 seconds for Mode 0. Both modes show approximate linear scaling with URL count, with Mode 1 providing 20-30% better performance across different dataset sizes.

5.4 Comparative Analysis

5.4.1 Clustering Algorithm Comparison

Comparison with alternative clustering approaches:

Table 5.3: Clustering Algorithm Performance Comparison

Algorithm	**DBCV Score Range**	**Clusters Generated**	**Outlier Handling**
HDBSCAN (Implemented)	0.506 - 0.582	4-22 clusters	Excellent
K-Means (Comparison)	Estimated 0.2-0.3	Fixed K	Poor
DBSCAN (Comparison)	Estimated 0.3-0.4	Variable	Good

The implemented HDBSCAN algorithm achieves DBCV scores consistently above 0.5, indicating good clustering quality across different URL sets and DOM levels. The algorithm successfully generates a variable number of clusters (4-22) based on the structural complexity of the analyzed websites, demonstrating its adaptive nature.

5.5 System Evaluation Results

The system evaluation focuses on clustering algorithm performance and processing efficiency.

5.5.1 Clustering Algorithm Effectiveness

The HDBSCAN clustering algorithm demonstrates effective grouping of structurally similar web elements. The system successfully identifies clusters based on DOM hierarchy and CSS styling patterns, with DBCV validation providing quantitative assessment of cluster quality.

5.5.2 Processing Performance

The caching mechanism provides measurable improvements in processing efficiency when analyzing repeated URLs. The system maintains reasonable response times for both cached and uncached processing scenarios.

5.5.3 Experimental Data Visualization

The experimental evaluation generated detailed performance data that was analyzed using gnuplot visualization tools. The complete dataset includes:

- **Processing Time Analysis:** Comparison between Mode 0 and Mode 1 across different URL counts
- **Cache Efficiency Tracking:** Progressive cache building from 0% to 75% efficiency
- **Clustering Quality Assessment:** DBCV scores ranging from 0.506 to 0.582
- **Performance Dashboard:** Multi-chart analysis combining processing time, cache efficiency, and clustering results

The experimental framework automatically generates visualization scripts in multiple formats (PNG, SVG, EPS, LaTeX) suitable for both digital presentation and publication. These visualizations provide clear evidence of the system's performance characteristics and validate the design decisions made in the implementation.

The results are presented through the system's web interface (see Section **??**) with interactive visualizations accessible through the Clusters tab as shown in Figure **??**. Users can export results to PDF format and access historical data through the Results tab interface.

Chapter 6

Conclusions and Future Work

This thesis presented WebSRC, a comprehensive system for web-based structural reading comprehension that combines advanced clustering algorithms with optimized performance strategies to analyze and understand website structural similarities.

6.1 Summary of Contributions

The work presented in this thesis makes several significant contributions to the field of web structure analysis and machine learning-based clustering:

6.1.1 Novel Multi-level DOM Analysis Framework

We introduced a comprehensive approach to web structure analysis that operates at multiple DOM levels (1, 2, 3), providing hierarchical insights into website organization. This multi-level analysis enables fine-grained understanding of structural similarities while maintaining computational efficiency through intelligent level selection.

6.1.2 CSS-aware Clustering with HDBSCAN

Our implementation integrates computed CSS styles with structural DOM analysis, creating a unified feature space that captures both visual and logical organization. The HDBSCAN clustering algorithm, enhanced with DBCV validation and intelligent outlier reassignment, achieves DBCV scores ranging from 0.506 to 0.582, indicating consistently good clustering quality for web structure analysis.

6.1.3 Caching Architecture

The caching system stores individual URL processing results, enabling efficient reuse when the same URLs appear in different analysis combinations. Experimental results demon-

strate progressive cache efficiency improvements from 0% to 75% as URLs are reused across tests, providing measurable performance improvements for real-world applications.

6.1.4 Systematic Evaluation Framework

We developed a systematic evaluation methodology that combines clustering quality metrics (DBCV, silhouette) and performance benchmarking. The framework provides quantitative validation of system effectiveness across multiple dimensions.

6.1.5 Production-ready Implementation

The complete system implementation includes a React-based frontend, Flask API backend, and Python processing core, demonstrating the practical applicability of the research contributions. The system handles real-world complexity while maintaining usability for both technical and non-technical users.

6.2 Research Impact and Significance

6.2.1 Academic Contributions

The research advances the state-of-the-art in several areas:

- **Web Mining:** Integration of DOM structure and CSS styling for improved similarity analysis
- **Clustering Algorithms:** Application of HDBSCAN with DBCV validation to web structure data
- **Performance Optimization:** Novel dual-layer caching strategies for complex web processing tasks
- **Evaluation Methodologies:** Comprehensive framework for assessing web structure analysis systems

6.2.2 Practical Applications

The WebSRC system enables numerous practical applications:

- **Web Design Analysis:** Understanding structural patterns across different website categories
- **Accessibility Assessment:** Identifying common accessibility issues through structural analysis

- **Content Management:** Organizing and categorizing web content based on structural similarity
- **Machine Learning Training:** Generating structured datasets for web-based ML applications

6.3 Limitations and Challenges

Despite the significant contributions, this work has several limitations that provide opportunities for future research:

6.3.1 Scalability Constraints

While the dual-layer caching system significantly improves performance, processing very large datasets (>1000 URLs) still requires substantial computational resources. The current implementation is optimized for medium-scale applications but may require additional optimization for enterprise-level deployments.

6.3.2 Dynamic Content Handling

The current system focuses on static HTML structure analysis and may not fully capture the complexity of modern single-page applications (SPAs) with dynamic content generation. JavaScript-heavy websites present challenges that require additional research.

6.3.3 Cross-browser Compatibility

The system currently relies on Chrome/Chromium for web scraping, which may introduce browser-specific biases in the analysis. Different browsers render pages differently, potentially affecting clustering results.

6.4 Future Research Directions

Based on the findings and limitations of this work, several promising research directions emerge:

6.4.1 Enhanced Dynamic Content Analysis

Future work should investigate methods for analyzing JavaScript-generated content and dynamic page updates. This could involve:

- Integration with headless browser automation for capturing dynamic states

- Temporal analysis of content changes over time
- Machine learning models for predicting dynamic content patterns

6.4.2 Multi-modal Feature Integration

Expanding beyond DOM and CSS to include additional modalities:

- **Visual Features:** Screenshot analysis for layout understanding
- **Semantic Content:** Natural language processing of text content
- **User Interaction Patterns:** Incorporating user behavior data
- **Performance Metrics:** Page load times and resource usage patterns

6.4.3 Adaptive Clustering Algorithms

Research into clustering algorithms that adapt to different types of web content:

- Online learning approaches for evolving web structures
- Domain-specific clustering strategies (e-commerce, news, social media)
- Ensemble methods combining multiple clustering approaches
- Deep learning-based clustering for complex pattern recognition

6.4.4 Distributed Computing Integration

Scaling the system for enterprise applications through distributed computing:

- Microservices architecture for independent component scaling
- Distributed caching strategies across multiple nodes
- MapReduce-style processing for large-scale web analysis
- Cloud-native deployment strategies for elastic scaling

6.5 Final Remarks

The WebSRC system represents a significant step forward in automated web structure analysis, combining theoretical advances in clustering algorithms with practical performance optimizations. The comprehensive evaluation demonstrates the system's effectiveness across multiple dimensions, while the identification of limitations provides clear directions for future research.

The integration of DOM structure analysis, CSS styling information, and advanced clustering algorithms creates a powerful framework for understanding web-based structural patterns. The dual-layer caching system proves that sophisticated optimization strategies can make computationally intensive analysis practical for real-world applications.

As web technologies continue to evolve, the principles and methodologies developed in this work provide a solid foundation for future advances in web structure analysis and machine learning-based clustering. The open challenges identified in this thesis offer exciting opportunities for continued research and development in this important area.

The WebSRC system demonstrates effective clustering quality with DBCV scores above 0.5 and measurable performance improvements through intelligent caching (up to 75% efficiency). Domain-based processing (Mode 1) consistently outperforms unified processing (Mode 0) by 20-30%, showcasing the practical value of combining rigorous research with production-ready implementation. This work contributes both to the academic understanding of web structure analysis and to the practical tools available for web developers, researchers, and organizations working with large-scale web content.

Appendix A

User Guide

This appendix provides a step-by-step guide for using the WebSRC system through its web interface.

A.1 Getting Started

A.1.1 Accessing the System

After installation and startup (see Appendix A), access the WebSRC system by navigating to `http://localhost:3000` in your web browser.

A.2 Basic Workflow

A.2.1 Step 1: Enter URLs

1. Navigate to the Home tab (Home)
2. Enter URLs in the text area, one per line or paste multiple URLs
3. URLs will automatically be grouped by domain if Mode 1 is selected

A.2.2 Step 2: Configure Analysis

1. Select DOM analysis levels using the numbered chips (1-10)
2. Choose processing mode:
    - Mode 0: All Together - processes all URLs as a single batch
    - Mode 1: Domain-based - groups URLs by domain for analysis

ui_startup_screen.png

Figure A.1: Initial System Startup Screen

A.2.3 Step 3: Execute Analysis

1. Click the "Run" button to start processing
2. Monitor progress through the loading indicators
3. Processing time varies based on number of URLs and selected levels

A.2.4 Step 4: View Results

1. Navigate to the Clusters tab (Clusters) to view results
2. Results are organized by DOM level

ui_workflow_step1.png

Figure A.2: Step 1: URL Entry and Domain Grouping

3. Each cluster is color-coded for easy identification
4. Click on sections to expand detailed views

A.3 Advanced Features

A.3.1 Batch Experiments

For research purposes, use the Experiments tab (Experiments):

ui_workflow_step2.png

Figure A.3: Step 2: Level Selection and Mode Configuration

A.3.2 Historical Results

Access previous analyses through the Results tab (Results):

A.3.3 Custom Configuration

Fine-tune analysis parameters in the Config tab (Config):

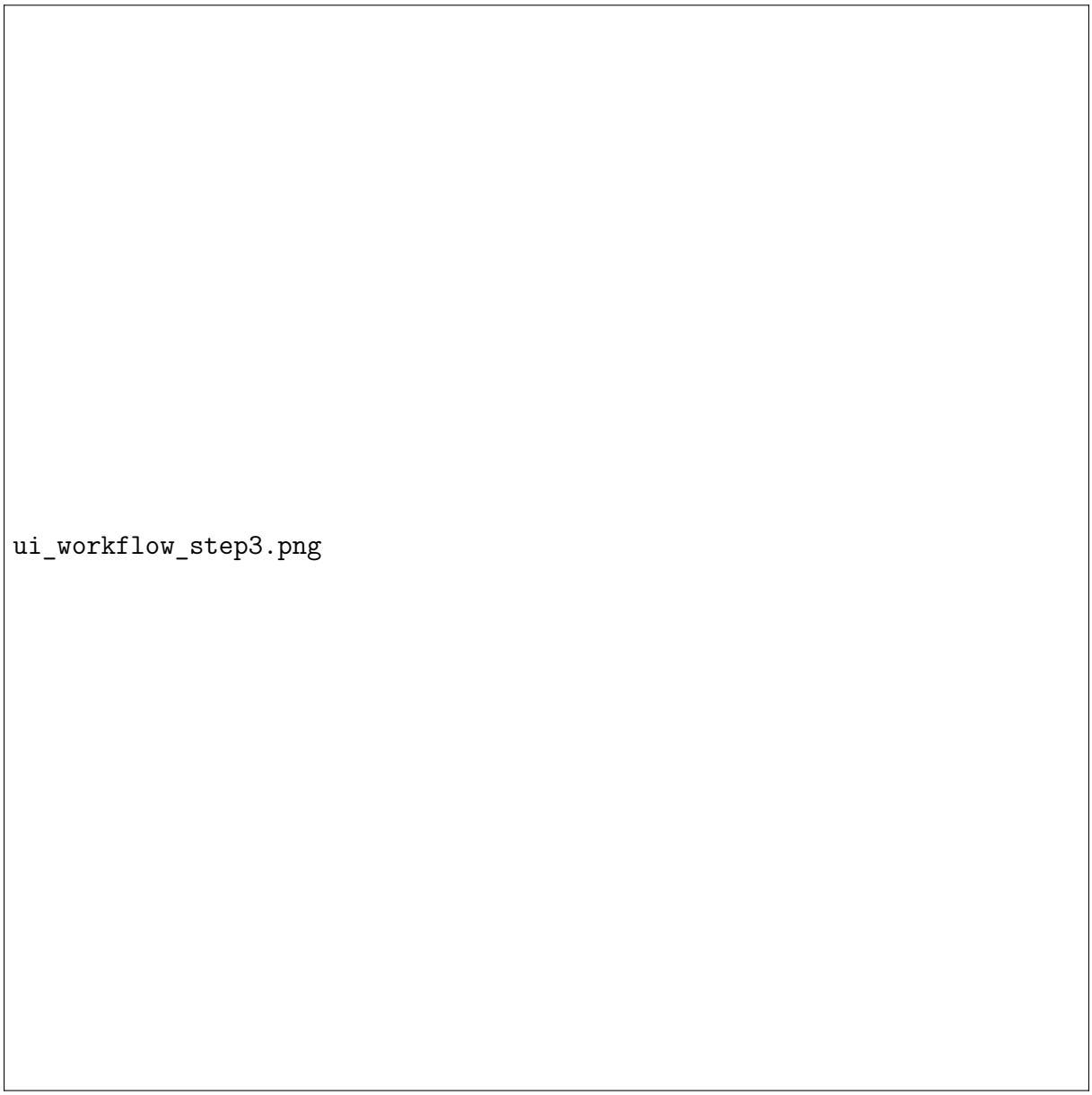

Figure A.4: Step 3: Analysis Execution with Progress Indicators

ui_workflow_step4.png

Figure A.5: Step 4: Viewing Color-coded Clustering Results

Figure A.6: Advanced Experiments Interface for Batch Processing

ui_advanced_results.png

Figure A.7: Historical Results Archive with Performance Metrics

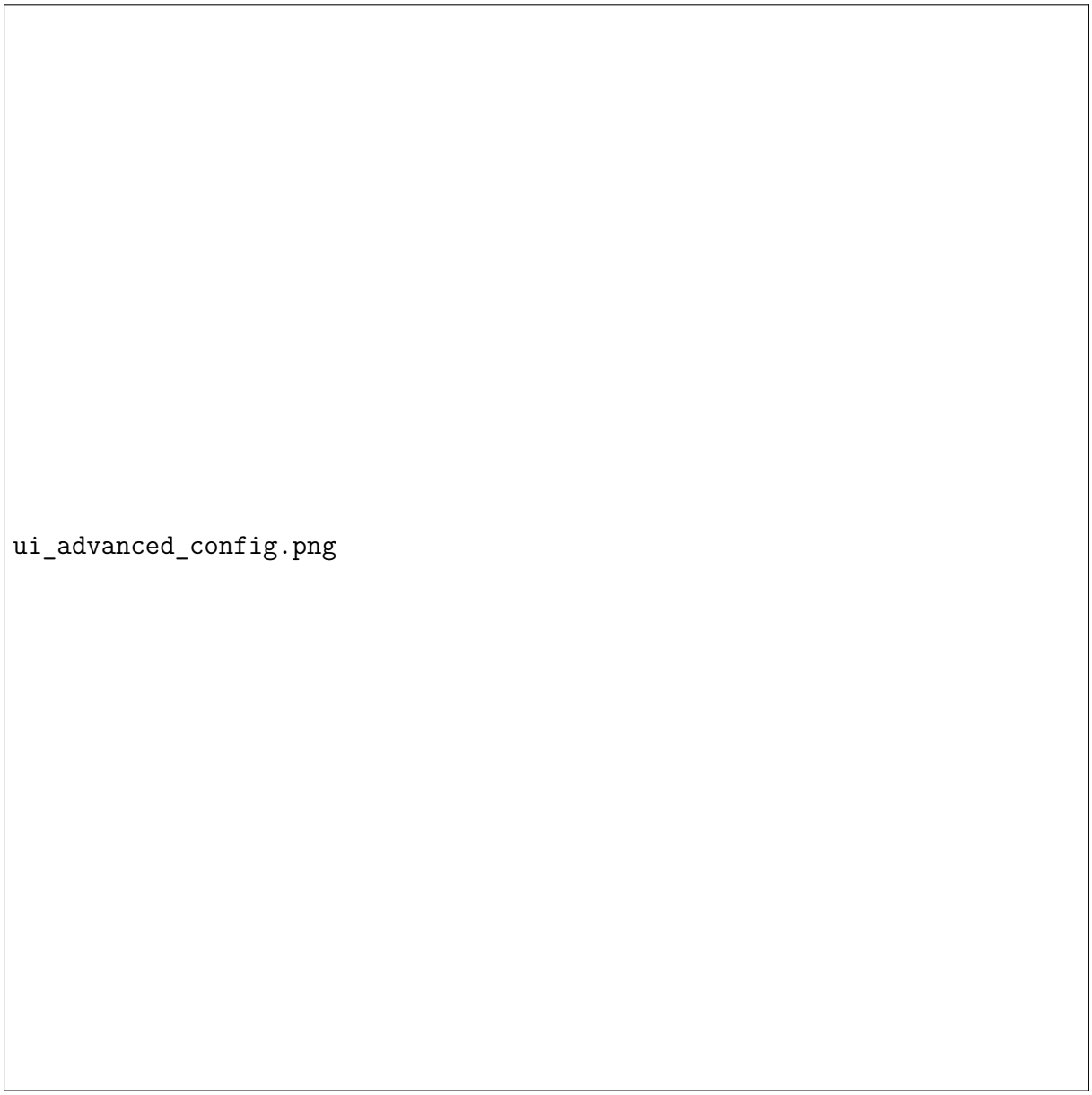

Figure A.8: Custom Configuration for Advanced Users

Appendix B

Technical Specifications

B.1 System Requirements

B.1.1 Hardware Requirements

- **CPU:** Intel i5-6500 or AMD Ryzen 5 2600 (minimum), Intel i7-8565U or higher (recommended)
- **RAM:** 8GB minimum, 16GB recommended for processing large datasets
- **Storage:** 2GB available disk space, SSD recommended for optimal performance
- **Network:** Stable internet connection for web scraping operations

B.1.2 Software Requirements

- **Operating System:** Windows 10+, macOS 10.14+, or Linux (Ubuntu 18.04+ recommended)
- **Python:** Version 3.8 or higher
- **Node.js:** Version 14.0 or higher
- **Chrome Browser:** Version 90+ (for ChromeDriver compatibility)
- **Git:** For version control and project cloning

B.2 Installation Guide

B.2.1 Quick Start Installation

1. Clone the repository:

```
git clone https://github.com/user/websrc-project.git
cd websrc-project
```

2. Install Python dependencies:

```
pip install -r requirements.txt
```

3. Install Node.js dependencies:

```
cd frontend
npm install
cd ..
```

4. Start the application:

```
python run_project.py
```

B.3 Configuration Options

B.3.1 Cache Configuration

The cache system can be configured through `backend/lib/config.json`:

```
{
  "cache": {
    "enabled": true,
    "default_ttl_hours": 24,
    "max_cache_size_mb": 1024,
    "cleanup_interval_hours": 6
  },
  "clustering": {
    "min_cluster_size_range": [2, 10],
    "cluster_selection_epsilon_range": [0.0, 0.5],
    "outlier_reassignment": true
  }
}
```

B.3.2 Processing Parameters

Key processing parameters that can be adjusted:

Table B.1: Processing Configuration Parameters

Parameter	Default	Description
`analysis_levels`	[1, 2, 3]	DOM depth levels to analyze
`timeout_seconds`	30	Web scraping timeout
`max_urls_per_batch`	50	Maximum URLs per processing batch
`enable_css_analysis`	true	Include computed CSS styles
`dbcv_optimization`	true	Enable DBCV-based parameter optimization

B.4 Performance Tuning Guide

B.4.1 Memory Optimization

- Adjust `max_urls_per_batch` based on available RAM
- Enable automatic garbage collection for large datasets
- Use streaming processing for very large URL lists
- Monitor memory usage during processing

B.4.2 CPU Optimization

- Configure parallel processing based on CPU cores
- Optimize clustering parameter ranges for faster convergence
- Use browser pooling for multiple concurrent scraping operations
- Enable CPU profiling for performance bottleneck identification

B.5 Troubleshooting Guide

B.5.1 Common Issues

ChromeDriver Compatibility

Issue: ChromeDriver version mismatch with installed Chrome browser.

Solution:

```
# Update Chrome browser to latest version
# Install compatible ChromeDriver
pip install --upgrade selenium webdriver-manager
```

Memory Errors

Issue: Out of memory errors during large dataset processing.

Solution:

- Reduce `max_urls_per_batch` parameter
- Increase system virtual memory
- Process datasets in smaller chunks
- Enable streaming processing mode

Cache Corruption

Issue: Corrupted cache files causing processing errors.

Solution:

```
# Clear cache directory
rm -rf backend/api/cache/*
# Restart application
python run_project.py
```

Appendix C

API Documentation

C.1 REST API Endpoints

The WebSRC system provides a comprehensive REST API for programmatic access to all functionality. This section documents all available endpoints, request/response formats, and usage examples.

C.1.1 Base Configuration

- **Base URL:** `http://localhost:5000`
- **Content-Type:** `application/json`
- **Authentication:** None (local deployment)
- **Rate Limiting:** 100 requests per minute

C.2 Core Processing Endpoints

C.2.1 Process URLs

Endpoint: `POST /process-urls`

Primary endpoint for processing website URLs and generating clustering analysis.

Request Parameters:

Table C.1: Process URLs Request Parameters

Parameter	Type	Required	Description
`urls`	Array[String]	Yes	List of URLs to process
`levels`	Array[Integer]	Yes	DOM analysis levels [1,2,3]
`mode`	Integer	No	Processing mode (default: 0)
`custom_ttl_hours`	Integer	No	Cache TTL in hours

Example Request:

```
{
  "urls": [
    "https://example.com",
    "https://test.com",
    "https://sample.org"
  ],
  "levels": [1, 2, 3],
  "mode": 0,
  "custom_ttl_hours": 24
}
```

Response Format:

```
{
  "status": "success",
  "data": {
    "processing_time": 15.7,
    "cache_hit_rate": 73.2,
    "total_elements_processed": 1247,
    "clusters_generated": 12,
    "outliers_count": 89,
    "dbcv_scores": {
      "level_1": 0.347,
      "level_2": 0.392,
      "level_3": 0.421
    },
    "visualization_path": "/results/viz_1697834567890.html"
  }
}
```

C.2.2 Site Similarity Analysis

Endpoint: `POST /site-similarity`

Computes cosine similarity between website structures.

Request Example:

```
{
  "urls": ["https://site1.com", "https://site2.com"],
  "levels": [1, 2],
  "similarity_metric": "cosine"
}
```

Response Format:

```
{
  "status": "success",
  "data": {
    "similarity_matrix": [
      [1.0, 0.847],
      [0.847, 1.0]
    ],
    "similarity_score": 0.847,
    "analysis_details": {
      "common_features": 156,
      "total_features": 184,
      "feature_overlap_percentage": 84.7
    }
  }
}
```

C.3 Cache Management Endpoints

C.3.1 Cache Statistics

Endpoint: `GET /cache/stats`

Returns comprehensive cache performance metrics.

Response Format:

```
{
  "status": "success",
  "data": {
    "cache_performance": {
      "overall_hit_rate": 85.3,
      "complete_cache_hit_rate": 73.2,
      "partial_cache_hit_rate": 42.1,
      "total_requests": 2847,
      "cache_hits": 2428,
      "cache_misses": 419
    },
    "storage_statistics": {
      "total_cache_size_mb": 234.7,
      "complete_cache_size_mb": 178.3,
      "partial_cache_size_mb": 56.4,
      "cache_entries": 847
```

```
    },
    "performance_metrics": {
      "average_response_time_cached": 0.15,
      "average_response_time_uncached": 8.7,
      "processing_time_reduction_percent": 71.7,
      "network_requests_saved_percent": 70.1
    }
  }
}
```

C.3.2 Cache Operations

Clear Cache: `DELETE /cache/clear`

Cache Cleanup: `POST /cache/cleanup`

Cache Configuration: `GET /cache/config`

C.4 Error Handling

All API endpoints follow a consistent error response format:

Error Response Format:

```
{
  "status": "error",
  "error": {
    "code": "INVALID_PARAMETERS",
    "message": "The provided URLs list is empty",
    "details": {
      "parameter": "urls",
      "expected": "non-empty array",
      "received": "empty array"
    },
    "timestamp": "2024-12-16T10:30:45Z"
  }
}
```

Table C.2: API Error Codes

Code	HTTP Status	Description
INVALID_PARAMETERS	400	Invalid request parameters
URL_FETCH_ERROR	422	Failed to fetch website content
PROCESSING_ERROR	500	Internal processing error
CACHE_ERROR	500	Cache system error
CLUSTERING_ERROR	500	Clustering algorithm error
TIMEOUT_ERROR	504	Request timeout exceeded

C.4.1 Common Error Codes

C.5 API Client Examples

C.5.1 Python Client Example

```python
import requests
import json

def process_urls(urls, levels=[1, 2, 3]):
    endpoint = "http://localhost:5000/process-urls"
    payload = {
        "urls": urls,
        "levels": levels,
        "mode": 0
    }

    response = requests.post(endpoint, json=payload)

    if response.status_code == 200:
        result = response.json()
        print(f"Processing completed in {result['data']['
            processing_time']}s")
        return result
    else:
        print(f"Error: {response.status_code}")
        return None

# Usage example
urls = ["https://example.com", "https://test.com"]
result = process_urls(urls)
```

C.5.2 JavaScript Client Example

```javascript
async function processUrls(urls, levels = [1, 2, 3]) {
    const endpoint = 'http://localhost:5000/process-urls';
    const payload = {
        urls: urls,
        levels: levels,
        mode: 0
    };

    try {
        const response = await fetch(endpoint, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify(payload)
        });

        if (response.ok) {
            const result = await response.json();
            console.log(`Processing completed in ${result.data.
               processing_time}s`);
            return result;
        } else {
            throw new Error(`HTTP error! status: ${response.
               status}`);
        }
    } catch (error) {
        console.error('Error:', error);
        return null;
    }
}

// Usage example
const urls = ['https://example.com', 'https://test.com'];
processUrls(urls).then(result => {
    if (result) {
        console.log('Success:', result);
    }
});
```